



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

14 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

WEEK 13 Lecture 2

CATCHING SIGNALS

INSTALLING SIGNAL HANDLERS

Installing Signal Handlers

- * The `signal` function modifies the default action associated with the receipt of signal `signum`:

```
#include <signal.h>
```

```
typedef void (*sighandler_t)(int);
```

```
    sighandler_t    signal(int signum, sighandler_t handler);
```

Returns: ptr to previous handler if OK, SIG_ERR on error (does not set errno)

- * The `signal` function can change the action associated with a signal `signum` in one of three ways:
 - If handler is **SIG_IGN**, then signals of type **signum** are ignored.
 - If handler is **SIG_DFL**, then the action for signals of type **signum** reverts to the default action.
 - Otherwise, handler is the address of a **user-defined function**, called a **signal handler**
 - * that will be called whenever the process receives a signal of type **signum**
 - * Changing the default action by passing the address of a handler to the `signal` function is known as **installing the handler**.
 - * The invocation of the handler is called **catching the signal**
 - * The execution of the handler is referred to as **handling the signal**.
 - * When the handler executes its return statement, control passes back to instruction in the control flow of the process that was interrupted by receipt of the signal

Installing Signal Handlers

- ★ When a process catches a signal of type k , the handler installed for signal k is invoked with a single integer argument set to k .
- ★ This argument allows the same handler function to catch different types of signals.
- ★ When the handler executes its return statement, control(usually) passes back to the instruction in the control flow where the process was interrupted by the receipt of the signal. We say “usually” because in some systems, interrupted system calls return immediately with an error.
- ★ **Signal handlers are yet another example of concurrency in a computer system.**
- ★ The execution of the signal handler interrupts the execution of the main C routine, akin to the way that a low-level exception handler interrupts the control flow of the current application program.
- ★ **Since the logical control flow of the signal handler overlaps the logical control flow of the main routine, the signal handler and the main routine run concurrently.**

Sending Signals with the alarm Function

- * A process can send SIGALRM signals to itself by calling the alarm function

```
#include <unistd.h>
```

```
unsigned int alarm(unsigned int secs);
```

Returns: remaining secs of previous alarm, or 0 if no previous alarm

- * The alarm function arranges for the kernel to send a SIGALRM signal to the calling process in secs seconds. If secs is zero, then no new alarm is scheduled. In any event, the call to alarm cancels any pending alarms, and returns the number of seconds remaining until any pending alarm was due to be delivered (had not this call to alarm canceled it), or 0 if there were no pending alarms.

```
void handler(int sig)
{
    static int beeps = 0;
    printf("BEEP\n");
    if (++beeps < 5)
        alarm(1);
    /* Next SIGALRM will be delivered in 1 second */
    else
    {
        printf("BOOM!\n");
        exit(0);
    }
}
```

```
int main()
{
    signal(SIGALRM, handler);
    /* Install SIGALRM handler */
    alarm(1);
    /* Next SIGALRM will be delivered in 1s */
    while (1) {
        ;
        /* Signal handler returns control here each time */
    }
    exit(0);
}
```

alarm.c

Sending Signals with the alarm Function

- ★ Figure shows a program called alarm that arranges to be interrupted by a SIGALRM signal every second for five seconds.
- ★ When the sixth SIGALRM is delivered it terminates.
- ★ When we run the program in Figure, we get the following output: a “BEEP” every second for five seconds, followed by a “BOOM” when the program terminates.

```
$ ./alarm
```

```
BEEP
```

```
BEEP
```

```
BEEP
```

```
BEEP
```

```
BEEP
```

```
BOOM!
```

- ★ Notice that the program in Figure uses the signal function to install a signal handler function (handler) that is called asynchronously, interrupting the infinite while loop in main, whenever the process receives a SIGALRM signal.
- ★ When the handler function returns, control passes back to main, which picks up where it was interrupted by the arrival of the signal.

Installing Signal Handlers

- * Figure shows a program that catches the SIGINT signal sent by the shell whenever the user types ctrl-c at the keyboard.
- * The default action for SIGINT is to immediately terminate the process.
- * In this example, we modify the default behavior to catch the signal, print a message, and then terminate the process.
- * The handler function is defined as shown in figure. The main routine installs the handler and then goes to sleep until a signal is received . When the SIGINT signal is received, the handler runs, prints a message, and then terminates the process.

```
void handler(int sig) /* SIGINT handler */
{
    printf("Caught SIGINT\n");
    exit(0);
}

int main()
{
    /* Install the SIGINT handler */
    if (signal(SIGINT, handler) == SIG_ERR)
        unix_error("signal error");
    pause(); /* Wait for the receipt of a signal */
    exit(0);
}
```

Signal Handling Example1

```
void sigint_handler(int sig) /* SIGINT handler */
{
    printf("So you think you can stop the bomb with ctrl-c, do you?\n");
    sleep(2);
    printf("Well...\n");
    fflush(stdout);
    sleep(1);
    printf("OK. :-)\n");
    exit(0);
}

int main()
{
    /* Install the SIGINT handler */
    if (signal(SIGINT, sigint_handler) == SIG_ERR)
        unix_error("signal error");

    /* Wait for the receipt of a signal */
    pause();

    return 0;
}
```

sigint.c

Signal Handling Example2

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

void handler1(int sig) /* SIGINT handler */
{
    if(sig == SIGINT)
        printf("Caught SIGINT\n");
}

void handler2(int sig) /* SIGQUIT handler */
{
    if(sig == SIGQUIT)
        printf("Caught SIGQUIT\n");
}
```

```
int main()
{
    /* Install the SIGINT handler */
    printf("Installed(Registered) SIGINT handler \n");
    if (signal(SIGINT, handler1) == SIG_ERR)
        unix_error("signal error");

    /* Install the SIGQUIT handler */
    printf("Installed(Registered) SIGQUIT handler \n");
    if (signal(SIGQUIT, handler2) == SIG_ERR)
        unix_error("signal error");

    printf("MAIN PROCESS NOW WAITING FOR signal \n");
    pause(); /* Wait for the receipt of a signal */

    exit(0);
}
```

sig2handle.c

SIGQUIT	→	Ctrl \
SIGINT	→	Ctrl c
SIGSTP	→	Ctrl z

Signal Handling Example3

```
include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

void handler(int sig) /* SIG handler */
{
    if(sig == SIGINT)
        printf("Caught SIGINT\n");
    else
        if(sig == SIGQUIT)
            printf("Caught SIGQUIT\n");
}
```

```
int main()
{
    /* Install the SIGINT handler */
    printf("Installed(Registered) SIGINT handler \n");
    if (signal(SIGINT, handler) == SIG_ERR)
        unix_error("signal error");

    /* Install the SIGQUIT handler */
    printf("Installed(Registered) SIGQUIT handler \n");
    if (signal(SIGQUIT, handler) == SIG_ERR)
        unix_error("signal error");

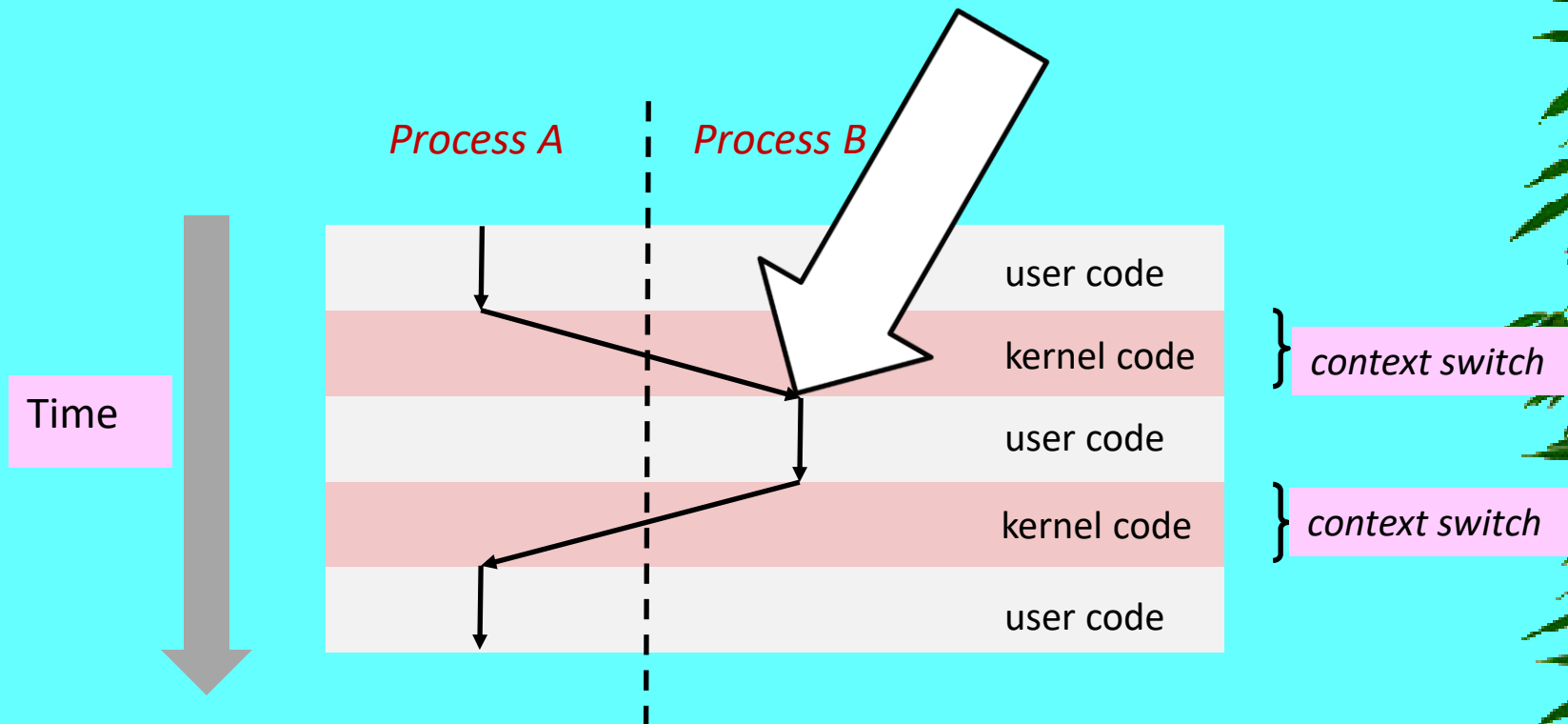
    printf("MAIN PROCESS NOW WAITING FOR signal \n");
    pause(); /* Wait for the receipt of a signal */

    exit(0);
}
sig2handle.c
```

SIGQUIT	→	Ctrl \
SIGINT	→	Ctrl c
SIGSTP	→	Ctrl z

Receiving Signals

- ★ Suppose kernel is returning from an exception handler and is ready to pass control to process B



Receiving Signals

- ✱ When the kernel is returning from an exception handler and is ready to pass control to process p, it checks the set of unblocked pending signals (pending & ~blocked) for process p.
- ✱ If this set is empty (the usual case), then the kernel passes control to the next instruction (Inext) in the logical control flow of p.
- ✱ However, if the set is nonempty, then the kernel chooses some signal k in the set (typically the smallest k) and forces p to receive signal k. The receipt of the signal triggers some action by the process.
- ✱ Once the process completes the action, then control passes back to then next instruction(Inext) in the logical control flow of p.
- ✱ **Each signal type has a predefined default action, which is one of the following:**
 - . **The process terminates.**
 - . **The process terminates and dumps core**
 - . **The process stops until restarted by a SIGCONT signal.**
 - . **The process ignores the signal.**

Receiving Signals

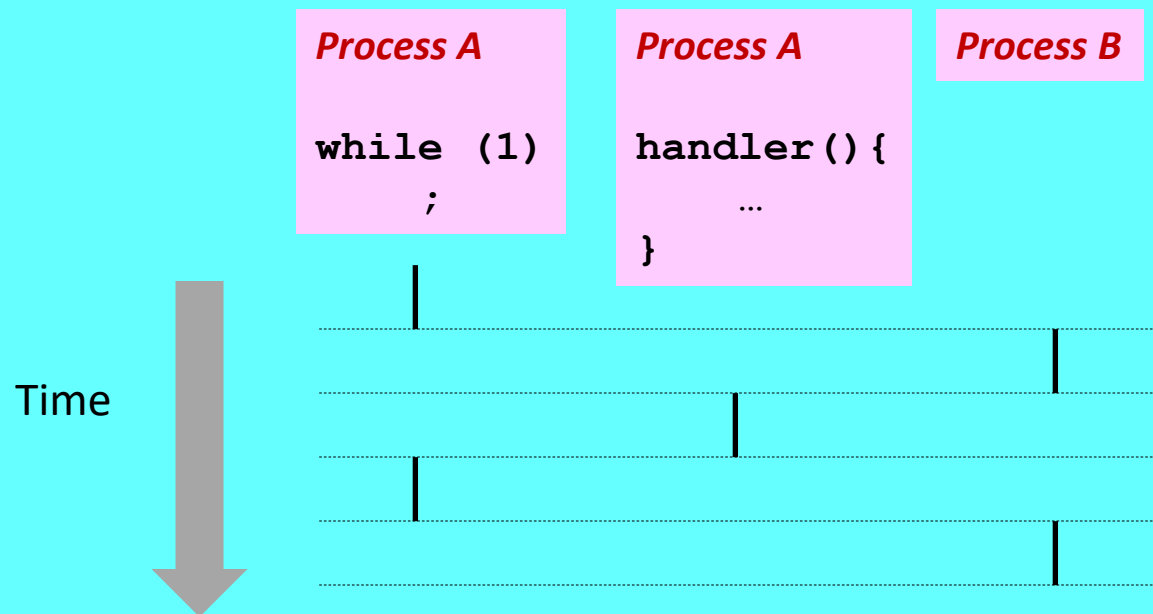
- * Suppose kernel is returning from an exception handler and is ready to pass control to process p
- * Kernel computes $pnb = pending \ \& \ \sim blocked$
 - The set of pending nonblocked signals for process p
- * If $(pnb == 0)$
 - Pass control to next instruction in the logical flow for p
- * Else
 - Choose least nonzero bit k in pnb and force process p to **receive** signal k
 - The receipt of the signal triggers some **action** by p
 - Repeat for all nonzero k in pnb
 - Pass control to next instruction in logical flow for p

Default Actions

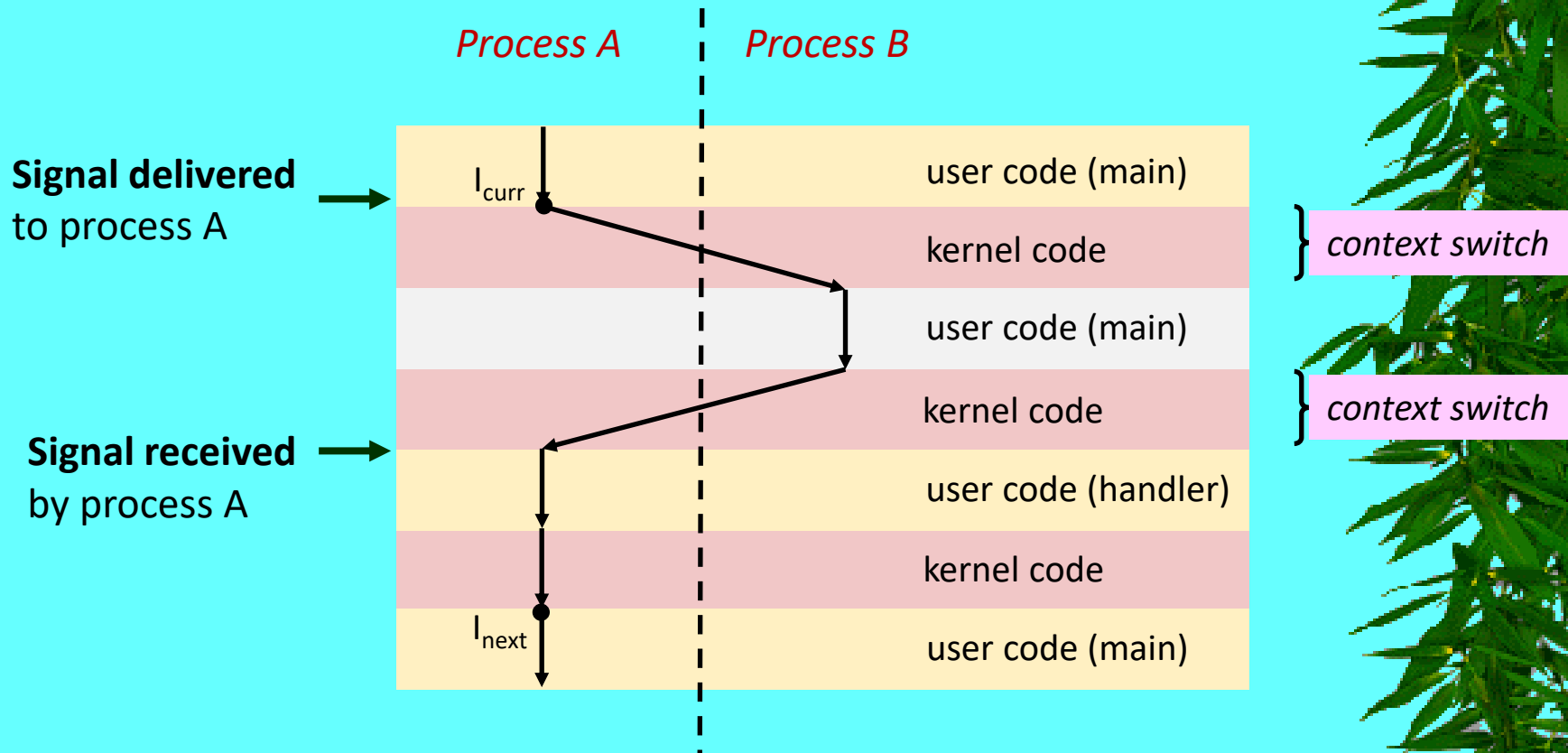
- ★ Each signal type has a predefined *default action*, which is one of:
 - The process terminates
 - The process stops until restarted by a SIGCONT signal
 - The process ignores the signal
- ★ For example, the default action for the receipt of a SIGKILL is to terminate the receiving process.
- ★ On the other hand, the default action for the receipt of a SIGCHLD is to ignore the signal.
- ★ A process can modify the default action associated with a signal by using the signal function.
- ★ The only exceptions are SIGSTOP and SIGKILL, whose default actions cannot be changed.

Signals Handlers as Concurrent Flows

- ★ A signal handler is a separate logical flow (not process) that runs concurrently with the main program

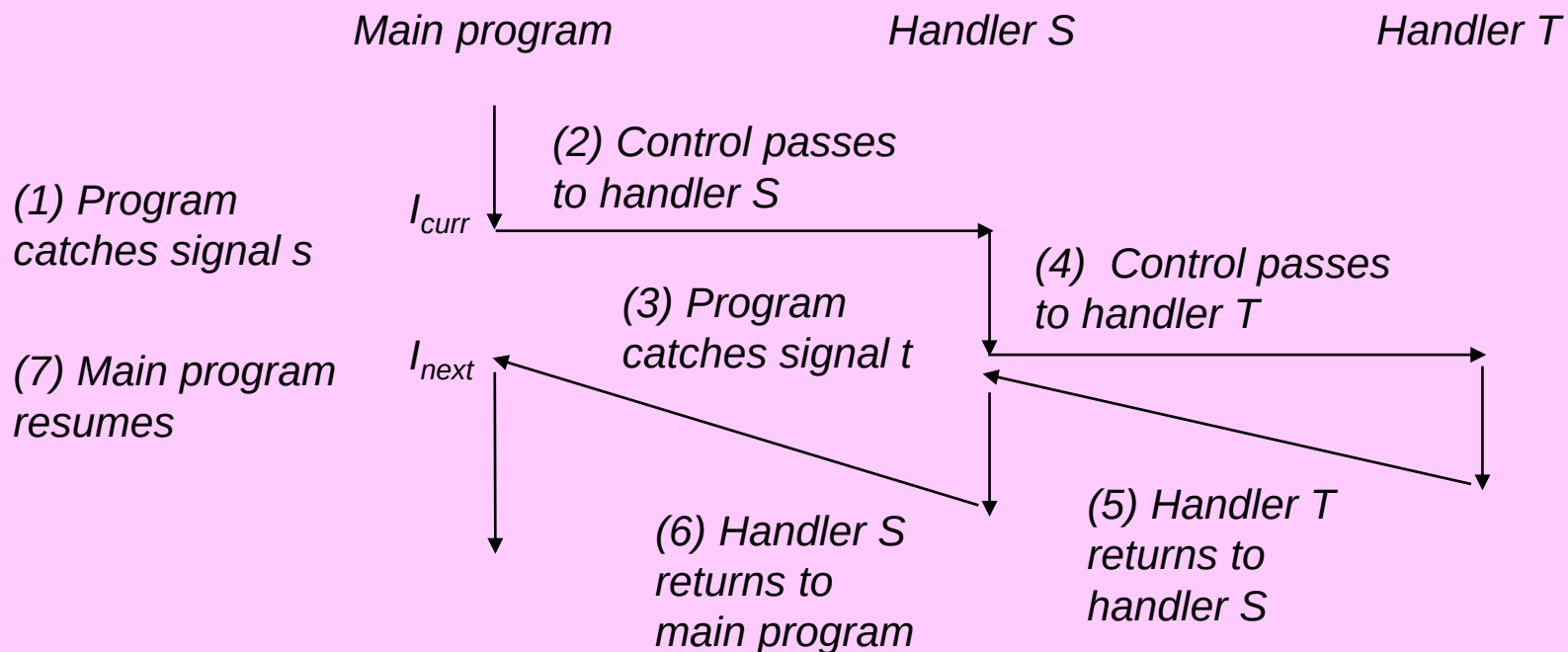


Another View of Signal Handlers as Concurrent Flows



Nested Signal Handlers

- ★ Handlers can be interrupted by other handlers



Blocking and Unblocking Signals

★ Implicit blocking mechanism

- Kernel blocks any pending signals of type currently being handled.
- E.g., A SIGINT handler can't be interrupted by another SIGINT

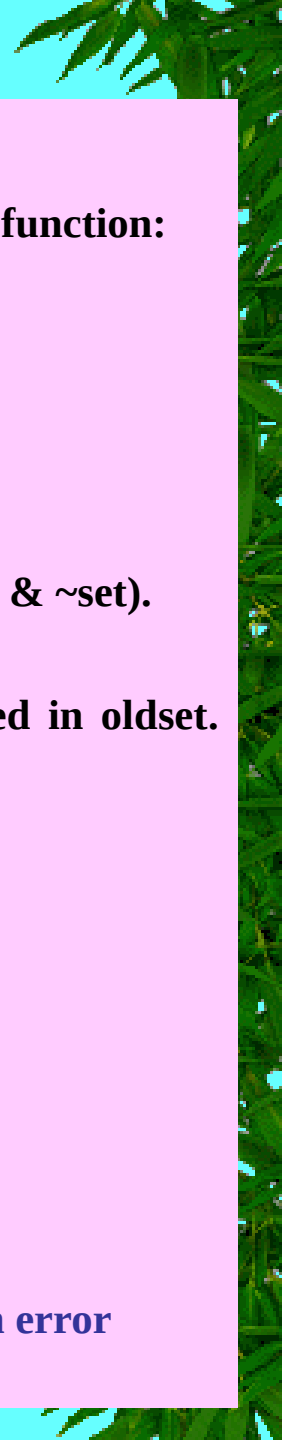
★ Explicit blocking and unblocking mechanism

- `sigprocmask` function

★ Supporting functions

- `sigemptyset` – Create empty set
- `sigfillset` – Add every signal number to set
- `sigaddset` – Add signal number to set
- `sigdelset` – Delete signal number from set

Blocking and Unblocking Signals



★ Explicit blocking and unblocking mechanism

- ★ Applications can explicitly block and unblock selected signals using the sigprocmask function:

```
#include <signal.h>
```

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

The sigprocmask function changes the set of currently blocked signals

The specific behavior depends on the value of how:

- . SIG_BLOCK: Add the signals in set to blocked (blocked = blocked | set).
- . SIG_UNBLOCK: Remove the signals in set from blocked (blocked = blocked & ~set).
- . SIG_SETMASK: blocked = set.

If oldset is non-NULL, the previous value of the blocked bit vector is stored in oldset.

Signal sets such as set are manipulated using the following functions:

```
int sigemptyset(sigset_t *set);
```

The sigemptyset initializes set to the emptyset

```
int sigfillset(sigset_t *set);
```

The sigfillset function adds every signal to set

```
int sigaddset(sigset_t *set, int signum);
```

The sigaddset function adds signum to set

```
int sigdelset(sigset_t *set, int signum); Returns: 0 if OK, -1 on error
```

sigdelset deletes signum from set

```
int sigismember(const sigset_t *set, int signum); Returns: 1 if member, 0 if not, -1 on error
```

sigismember returns 1 if signum is a member of set, and 0 if not.

Temporarily Blocking Signals

```
sigset_t mask, prev_mask;

Sigemptyset(&mask);
Sigaddset(&mask, SIGINT);

/* Block SIGINT and save previous blocked set */
Sigprocmask(SIG_BLOCK, &mask, &prev_mask);

/* Code region that will not be interrupted by SIGINT */

/* Restore previous blocked set, unblocking SIGINT */
Sigprocmask(SIG_UNBLOCK, &mask, &prev_mask);
```

SOC 2040

SYSTEM PROGRAMMING

SAFE SIGNAL HANDLING

Safe Signal Handling

- ★ Handlers are tricky because they are concurrent with main program and share the same global data structures.
 - Shared data structures can become corrupted.
- ★ For now here are some guidelines to help you avoid trouble.

Async-Signal-Safety

- ★ Function is *async-signal-safe* if either reentrant (e.g., all variables stored on stack frame,) or non-interruptible by signals.
- ★ Posix guarantees 117 functions to be async-signal-safe
 - Source: “man 7 signal”
 - Popular functions on the list:
 - ★ `_exit`, `write`, `wait`, `waitpid`, `sleep`, `kill`
 - Popular functions that are **not** on the list:
 - ★ `printf`, `sprintf`, `malloc`, `exit`
 - ★ Unfortunate fact: `write` is the only async-signal-safe output function

Guidelines for Writing Safe Handlers

- ✱ G0: Keep your handlers as simple as possible
 - e.g., Set a global flag and return
- ✱ G1: Call only async-signal-safe functions in your handlers
 - `printf`, `sprintf`, `malloc`, and `exit` are not safe!
- ✱ G2: Save and restore `errno` on entry and exit
 - So that other handlers don't overwrite your value of `errno`
- ✱ G3: Protect accesses to shared data structures by temporarily blocking all signals.
 - To prevent possible corruption
- ✱ G4: Declare global variables as `volatile`
 - To prevent compiler from storing them in a register
- ✱ G5: Declare global flags as `volatile sig_atomic_t`
 - *flag*: variable that is only read or written (e.g. `flag = 1`, not `flag++`)
 - Flag declared this way does not need to be protected like other globals

SOC 2040

SYSTEM PROGRAMMING

NON-LOCAL JUMPS

Nonlocal Jumps: `setjmp/longjmp`

- ★ Powerful (but dangerous) user-level mechanism for transferring control to an arbitrary location
 - Controlled to way to break the procedure call / return discipline
 - Useful for error recovery and signal handling
- ★ `int setjmp(jmp_buf j)`
 - Must be called before `longjmp`
 - Identifies a return site for a subsequent `longjmp`
 - Called **once**, returns **one or more** times
- ★ Implementation:
 - Remember where you are by storing the current **register context**, **stack pointer**, and **PC value** in `jmp_buf`
 - Return 0

setjmp/longjmp (cont)

- * `void longjmp(jmp_buf j, int i)`
 - Meaning:
 - * return from the `setjmp` remembered by jump buffer `j` again ...
 - * ... this time returning `i` instead of 0
 - Called after `setjmp`
 - Called **once**, but **never** returns
- * **longjmp Implementation:**
 - Restore register context (stack pointer, base pointer, PC value) from jump buffer `j`
 - Set `%eax` (the return value) to `i`
 - Jump to the location indicated by the PC stored in jump buf `j`

setjmp/longjmp Example

- ★ Goal: return directly to original caller from a deeply-nested function

```
/* Deeply nested function foo */  
void foo(void)  
{  
    if (error1)  
        longjmp(buf, 1);  
    bar();  
}  
  
void bar(void)  
{  
    if (error2)  
        longjmp(buf, 2);  
}
```

setjmp/longjmp Example (cont)

```
jmp_buf buf;

int error1 = 0;
int error2 = 1;

void foo(void), bar(void);

int main()
{
    switch(setjmp(buf)) {
        case 0:
            foo();
            break;
        case 1:
            printf("Detected an error1 condition in foo\n");
            break;
        case 2:
            printf("Detected an error2 condition in foo\n");
            break;
        default:
            printf("Unknown error condition in foo\n");
    }
    exit(0);
}
```

```
/* Deeply nested function foo */
void foo(void)
{
    if (error1)
        longjmp(buf, 1);
    bar();
}

void bar(void)
{
    if (error2)
        longjmp(buf, 2);
}
```

Putting It All Together: A Program That Restarts Itself When `ctrl-c`'d

```
jmp_buf buf;

void handler(int sig)
{
    longjmp(buf, 1);
}

int main()
{
    if (!setjmp(buf)) {
        Signal(SIGINT, handler);
        puts("starting\n");
    }
    else
        puts("restarting\n");

    while(1) {
        sleep(1);
        puts("processing...\n");
    }
    exit(0); /* Control never reaches here */
}
```

restart.c

```
$/restart
starting
processing...
processing...
processing...
restarting ←
processing...
processing...
restarting ←
processing...
processing...
processing...
```

Ctrl-c

Ctrl-c

Tools for Manipulating Processes

Linux systems provide a number of useful tools for monitoring and manipulating processes:

- ★ **strace:** Prints a trace of each system call invoked by a running program and its children. A fascinating tool for the curious student. Compile your program with `-static` to get a cleaner trace without a lot of output related to shared libraries.
- ★ **ps:** Lists processes (including zombies) currently in the system.
- ★ **top:** Prints information about the resource usage of current processes.
- ★ **pmap:** Displays the memory map of a process.
- ★ **/proc:** A virtual filesystem that exports the contents of numerous kernel data structures in an ASCII text form that can be read by user programs.

For example, type `cat /proc/loadavg` to see the current load average on your Linux system

Summary

- ★ Signals provide process-level exception handling
 - Can generate from user programs
 - Can define effect by declaring signal handler
 - Be very careful when writing signal handlers
- ★ Nonlocal jumps provide exceptional control flow within process
 - Within constraints of stack discipline

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 8 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

14 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 13 LECTURE 2

CONCURRENT PROGRAMMING

Concurrent Programming **BASICS**

Concurrent Programming is Hard!

- ★ The human mind tends to be sequential
- ★ The notion of time is often misleading
- ★ Thinking about all possible sequences of events in a computer system is at least error prone and frequently impossible

Concurrent Programming

- ★ Logical control flows are concurrent if they overlap in time.
- ★ This general phenomenon, known as concurrency, shows up at many different levels of a computer system.
- ★ Hardware exception handlers, processes, and Unix signal handlers are all familiar examples.

Concurrent Programming

- ★ Concurrency is considered mainly as a mechanism that the operating system kernel uses to run multiple application programs.
- ★ But concurrency is not just limited to the kernel.
- ★ It can play an important role in application programs as well.
- ★ For example, Unix signal handlers allow applications to respond to asynchronous events such as the user typing ctrl-c or the program accessing an undefined area of virtual memory.
- ★ **Application-level concurrency is useful in other ways as well:**
 - **Accessing slow I/O devices**
 - **Interacting with humans**
 - **Reducing latency by deferring work**
 - **Servicing multiple network clients**
 - **Computing in parallel on multi-core machines**

Concurrent Programming

- ★ Applications that use application-level concurrency are known as **concurrent programs**.
- ★ Modern operating systems provide three basic approaches for building concurrent programs:
 - Processes
 - I/O Multiplexing
 - Threads

Concurrent Programming

❑ Processes

- ★ With this approach, each logical control flow is a process that is scheduled and maintained by the kernel.
- ★ Since processes have separate virtual address spaces, flows that want to communicate with each other must use some kind of explicit inter-process communication (IPC) mechanism.

Concurrent Programming

❑ I/O multiplexing

- ★ This is a form of concurrent programming where applications explicitly schedule their own logical flows in the context of a single process.
- ★ Logical flows are modelled as state machines that the main program explicitly transitions from state to state as a result of data arriving on file descriptors.
- ★ Since the program is a single process, all flows share the same address space.

Example : Event-driven Network Client-server

Concurrent Programming

Concurrent Programming

❑ Threads

- ★ Threads are logical flows that run in the context of a single process and are scheduled by the kernel.
- ★ You can think of threads as a hybrid of the other two approaches, scheduled by the kernel like process flows, and sharing the same virtual address space like I/O multiplexing flows.

Concurrent Programming is Hard!

★ Classical problem classes of concurrent programs:

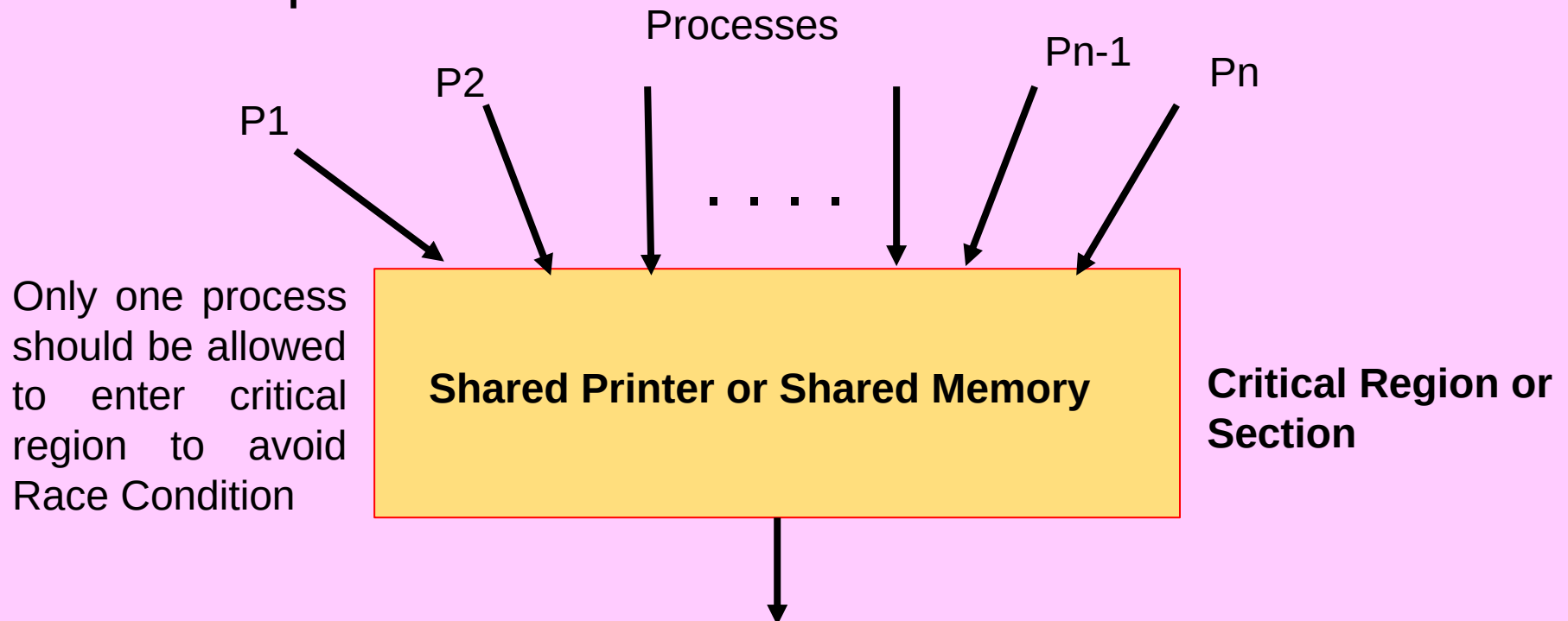
- *Races*

- *Deadlock*

- *Livelock / Starvation / Fairness*

Concurrent Programming is Hard!

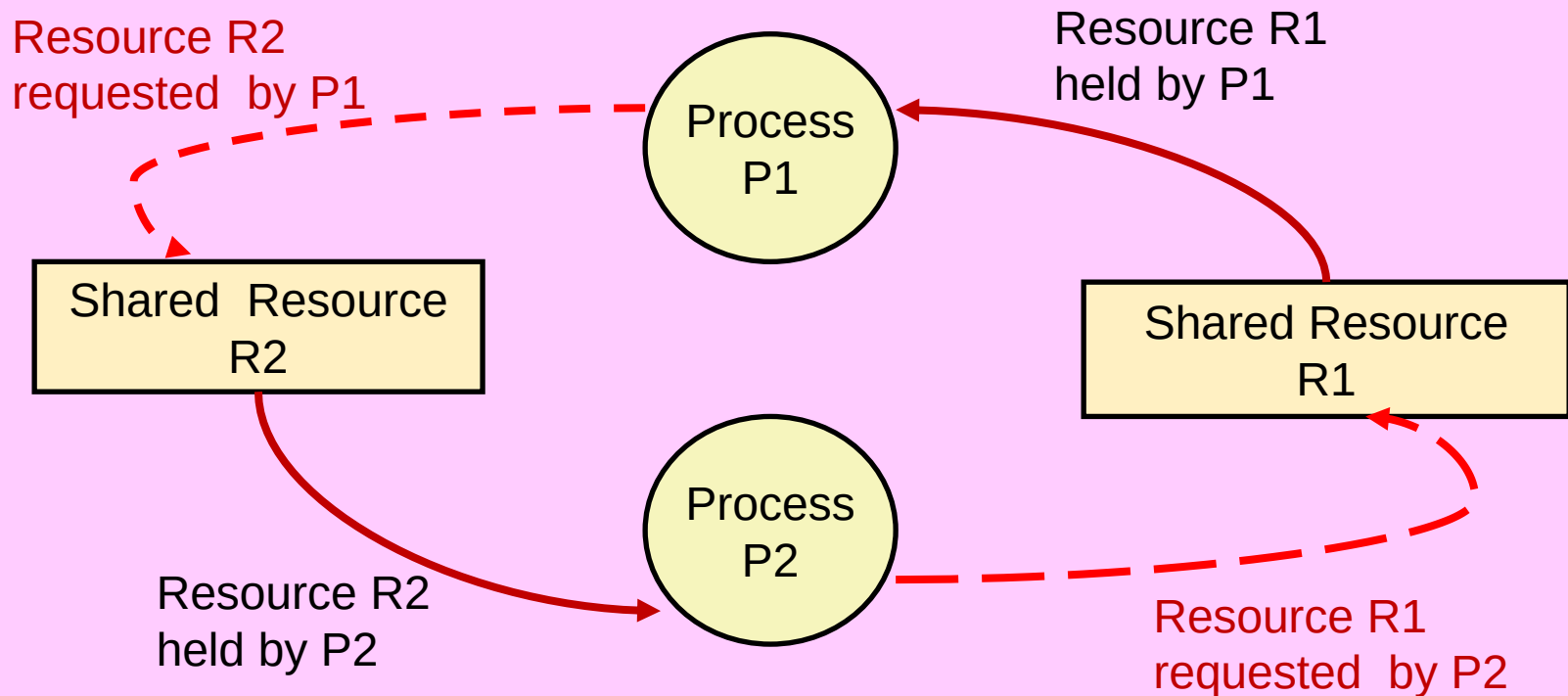
- ★ Classical problem classes of concurrent programs:
 - **Races:** outcome depends on arbitrary scheduling decisions elsewhere in the system
- ★ Example: who gets the last seat on the airplane?



Concurrent Programming is Hard!

- ★ Classical problem classes of concurrent programs:
 - **Deadlock:** improper resource allocation prevents forward progress

- ★ Example: traffic gridlock

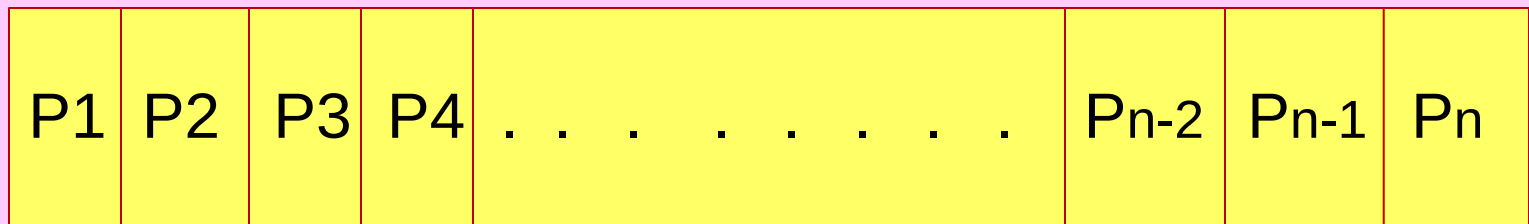


Concurrent Programming is Hard!

- ★ Classical problem classes of concurrent programs:
 - **Livelock / Starvation / Fairness**: external events and/or system scheduling decisions can prevent sub-task progress
 - ★ Example: people always jump in front of you in line or queue

Higher Priority Processes

Lower Priority Processes



READY QUEUE OR RUNQUEUE

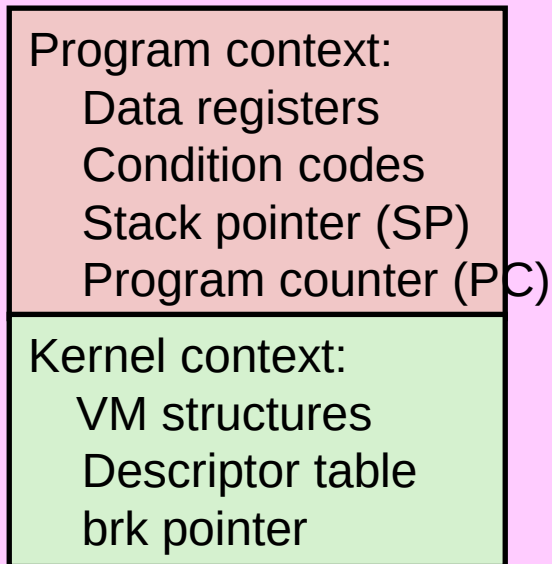
Processes Starving

Process Scheduler

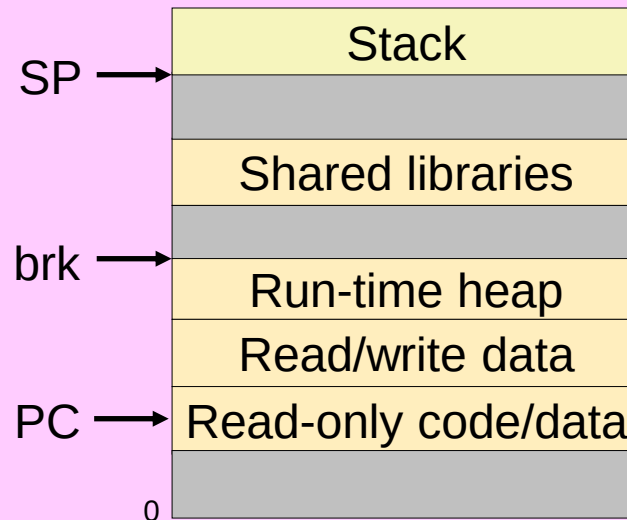
Traditional View of a Process

- ★ Process = process context + code, data, and stack

Process context

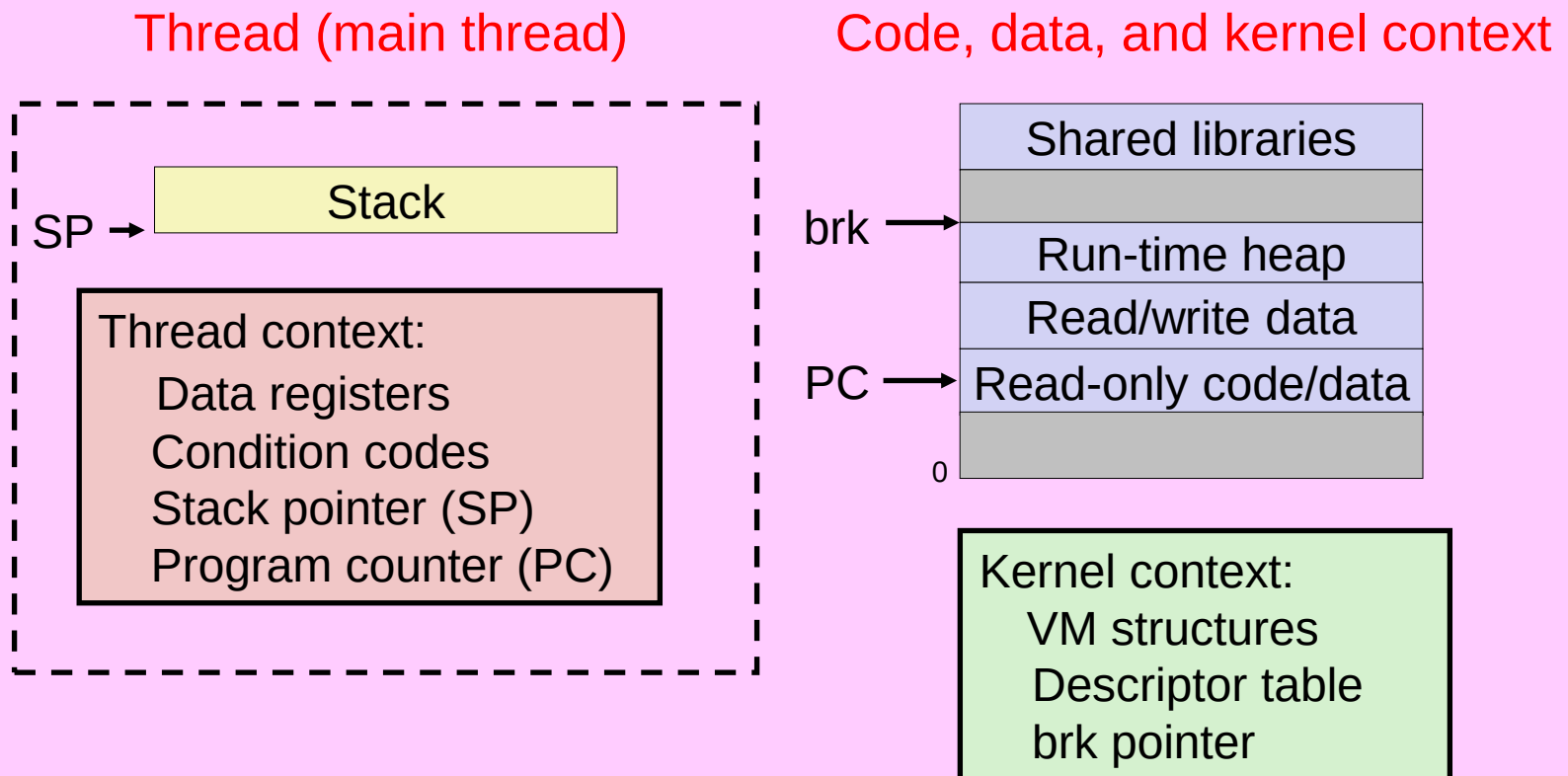


Code, data, and stack



Alternate View of a Process

- ★ Process = thread + code, data, and kernel context



A Process With Multiple Threads

- ★ Multiple threads can be associated with a process
 - Each thread has its own logical control flow
 - Each thread shares the same code, data, and kernel context
 - Each thread has its own stack for local variables
 - ★ but not protected from other threads
 - Each thread has its own thread id (TID)

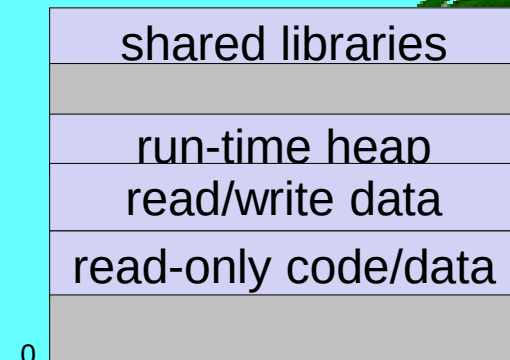
Thread 1 (main thread) Thread 2 (peer thread) Shared code and data

stack 1

stack 2

Thread 1 context:
Data registers
Condition codes
SP1
PC1

Thread 2 context:
Data registers
Condition codes
SP2
PC2

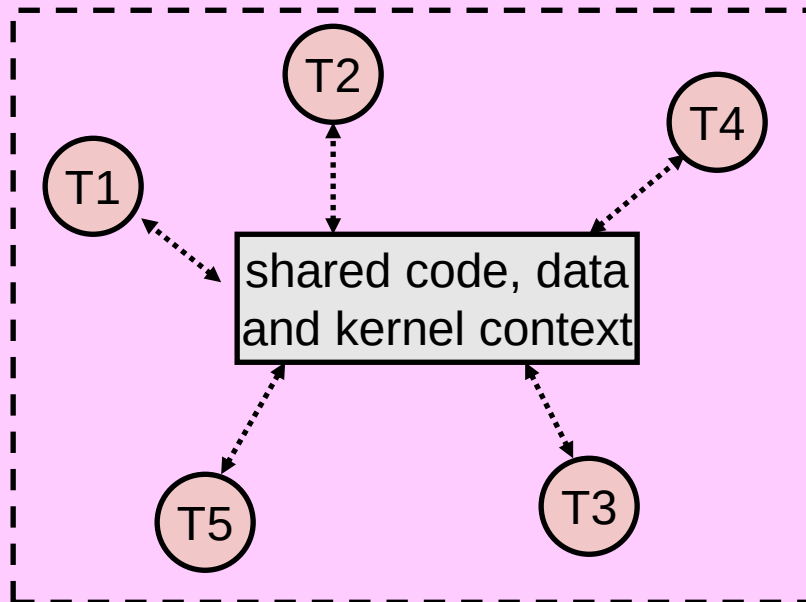


Kernel context:
VM structures
Descriptor table
brk pointer⁵⁰

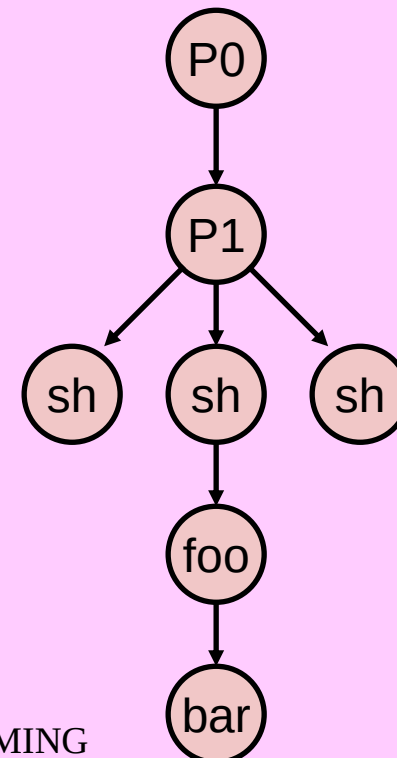
Logical View of Threads

- ★ Threads associated with process form a pool of peers
 - Unlike processes which form a tree hierarchy

Threads associated with process foo

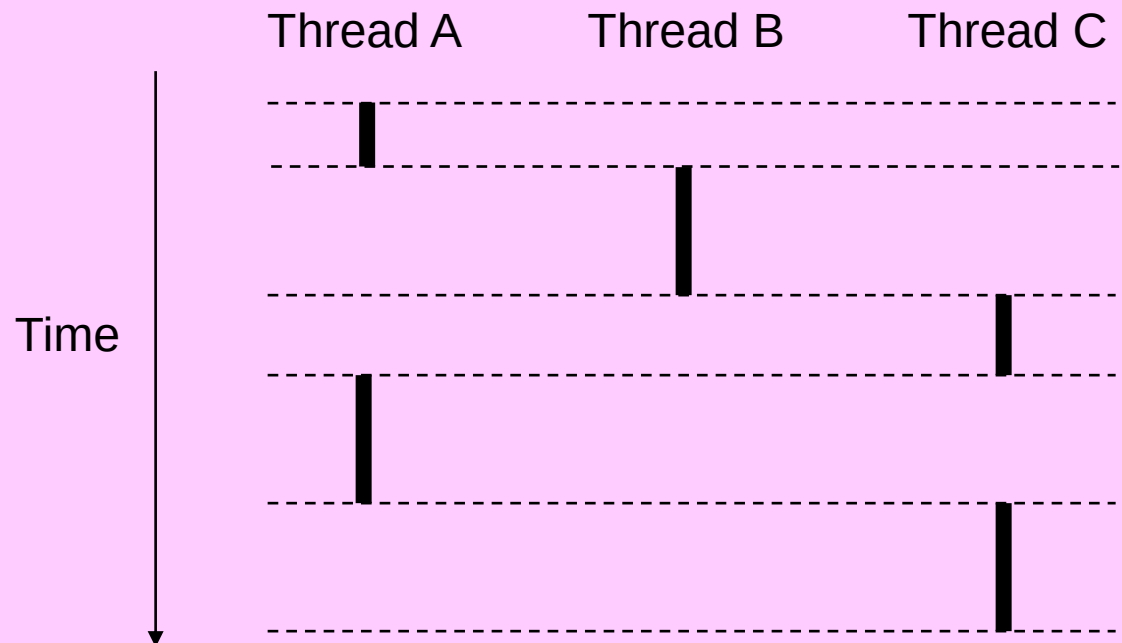


Process hierarchy



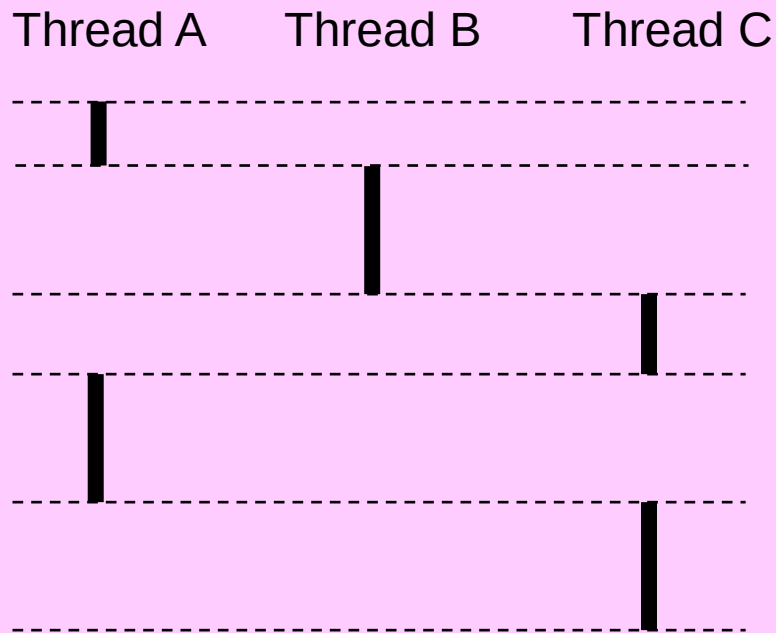
Concurrent Threads

- ★ Two threads are *concurrent* if their flows overlap in time
- ★ Otherwise, they are sequential
- ★ Examples:
 - Concurrent: A & B, A&C
 - Sequential: B & C



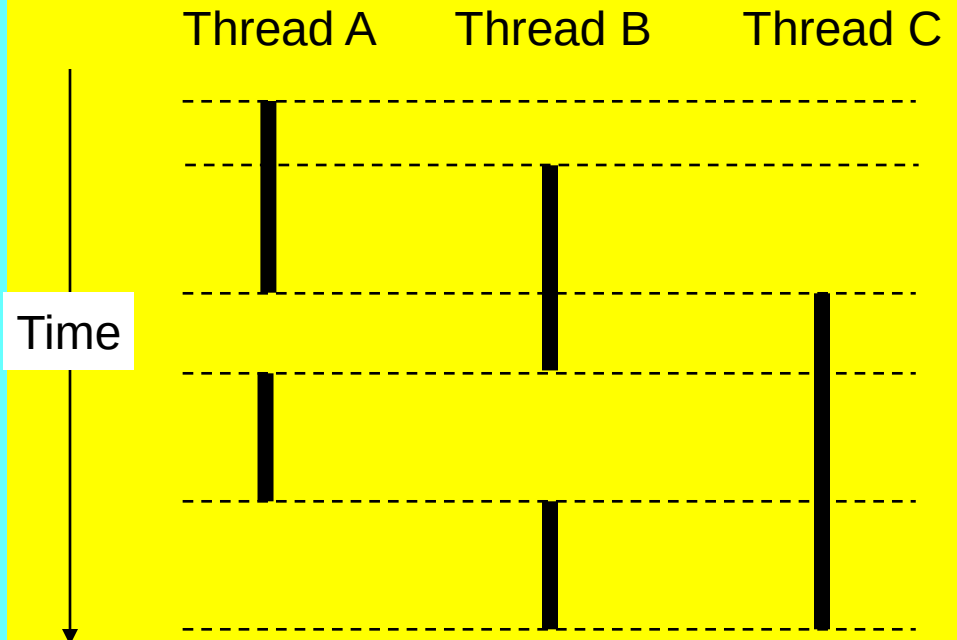
Concurrent Thread Execution

- ★ Single Core Processor
 - Simulate parallelism by time slicing



14 May 2024

- ★ Multi-Core Processor
 - Can have true parallelism



Run 3 threads on 2 cores

Threads vs. Processes

- ★ How threads and processes are similar
 - Each has its own logical control flow
 - Each can run concurrently with others (possibly on different cores)
 - Each is context switched
- ★ How threads and processes are different
 - Threads share all code and data (except local stacks)
 - ★ Processes (typically) do not
 - Threads are somewhat less expensive than processes
 - ★ Process control (creating and reaping) twice as expensive as thread control
 - ★ Linux numbers:
 - ~20K cycles to create and reap a process
 - ~10K cycles (or less) to create and reap a thread

Concurrent Programming with Threads

- ★ A thread is a logical flow that runs in the context of a process.
- ★ Normally programs consist of a single thread per process.
- ★ But modern systems also allow us to write programs that have multiple threads running concurrently in a single process.
- ★ The threads are scheduled automatically by the kernel.
- ★ Each thread has its own thread context, including an unique integer thread ID (TID), stack, stack pointer, program counter, general-purpose registers, and condition codes.
- ★ All threads running in a process share the entire virtual address space of that process.

Concurrent Programming with Threads

- ★ Logical flows based on threads combine qualities of flows based on processes and I/O multiplexing.
- ★ Like processes, threads are scheduled automatically by the kernel and are known to the kernel by an integer ID.
- ★ Like flows based on I/O multiplexing, multiple threads run in the context of a single process, and thus share the entire contents of the process virtual address space, including its code, data, heap, shared libraries, and open files

Posix Threads (Pthreads) Interface

- * *Pthreads*: Standard interface for ~60 functions that manipulate threads from C programs
 - Creating and reaping threads
 - * `pthread_create()`
 - * `pthread_join()`
 - Determining your thread ID
 - * `pthread_self()`
 - Terminating threads
 - * `pthread_cancel()`
 - * `pthread_exit()`
 - * `exit()` [terminates all threads] , `RET` [terminates current thread]
 - Synchronizing access to shared variables
 - * `pthread_mutex_init`
 - * `pthread_mutex_[un]lock`

Thread Execution Model



❑ Creating Threads

- ★ Threads create other threads by calling the `pthread_create` function.

```
#include <pthread.h>
typedef void *(func)(void *);
```

```
int pthread_create(pthread_t *tid, pthread_attr_t *attr, func *f, void *arg);
```

Returns: 0 if OK, nonzero on error

- ★ The `pthread_create` function creates a new thread and runs the thread routine `f` in the context of the new thread and with an input argument of `arg`.
- ★ The `attr` argument can be used to change the default attributes of the newly created thread.
- ★ Normally `pthread_create` is called with a `NULL` `attr` argument.
- ★ When `pthread_create` returns, argument `tid` contains the ID of the newly created thread.
- ★ The new thread can determine its own thread ID by calling the `pthread_self` function.

```
#include <pthread.h>
```

```
pthread_t pthread_self(void);
```

Returns: thread ID of caller

Thread Execution Model



❑ Terminating Threads

★ A thread terminates in one of the following ways:

- The thread terminates implicitly when its top-level thread routine returns.
- The thread terminates explicitly by calling the `pthread_exit` function. If the main thread calls `pthread_exit`, it waits for all other peer threads to terminate, and then terminates the main thread and the entire process with a return value of `thread_return`.

```
#include <pthread.h>
void pthread_exit(void *thread_return);
```

- ★ Some peer thread calls the Unix `exit` function, which terminates the process and all threads associated with the process.
- ★ Another peer thread terminates the current thread by calling the `pthread_cancel` function with the ID of the current thread.

```
#include <pthread.h>
int pthread_cancel(pthread_t tid)
```

Returns: 0 if OK, nonzero on error

Thread Execution Model



❑ Reaping Terminated Threads

- ★ Threads wait for other threads to terminate by calling the `pthread_join` function.

```
#include <pthread.h>
```

```
int pthread_join(pthread_t tid, void **thread_return)
```

Returns: 0 if OK, nonzero on error

- ★ The `pthread_join` function blocks until thread `tid` terminates, assigns the generic(`void *`) pointer returned by the thread routine to the location pointed to by `thread_return`, and then reaps any memory resources held by the terminated thread.
- ★ Notice that, unlike the Unix `wait` function, the `pthread_join` function can only wait for a specific thread to terminate.
- ★ There is no way to instruct `pthread_wait` to wait for an arbitrary thread to terminate.
- ★ This can complicate our code by forcing us to use other, less intuitive mechanisms to detect process termination.

Thread Execution Model



❑ Detaching Threads

- ★ At any point in time, a thread is joinable or detached.
- ★ A joinable thread can be reaped and killed by other threads.
- ★ Its memory resources (such as the stack) are not freed until it is reaped by another thread.
- ★ In contrast, a detached thread cannot be reaped or killed by other threads.
- ★ Its memory resources are freed automatically by the system when it terminates.
- ★ By default, threads are created joinable.
- ★ In order to avoid memory leaks, each joinable thread should either be explicitly reaped by another thread, or detached by a call to the `pthread_detach` function.

```
#include <pthread.h>
```

```
int pthread_detach(pthread_t tid);
```

Returns: 0 if OK, nonzero on error

- ★ The `pthread_detach` function detaches the joinable thread `tid`.
- ★ Threads can detach themselves by calling `pthread_detach` with an argument of `pthread_self()`.

Thread Execution Model



□ Initializing Threads

- * The `pthread_once` function allows you to initialize the state associated with a thread routine.

```
#include <pthread.h>
```

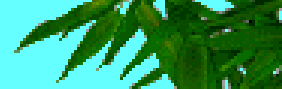
```
pthread_once_t once_control = PTHREAD_ONCE_INIT;
```

```
int pthread_once(pthread_once_t *once_control, void (*init_routine)(void));
```

Always returns 0

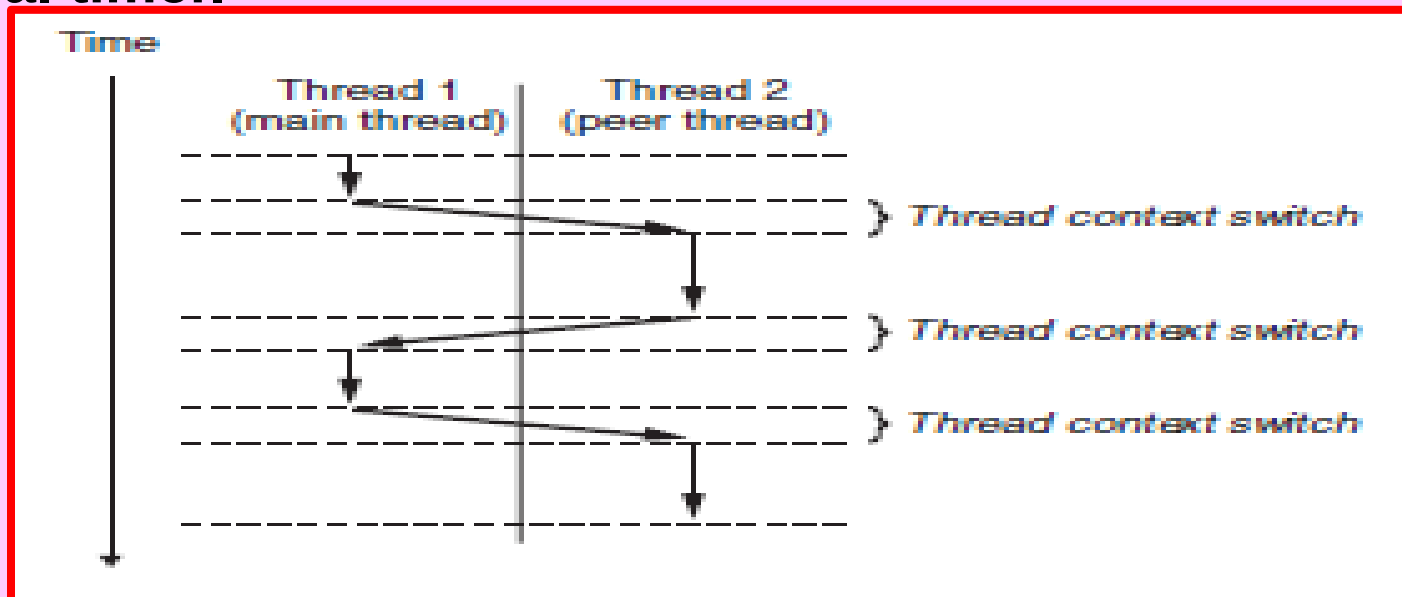
- * The `once_control` variable is a global or static variable that is always initialized to `PTHREAD_ONCE_INIT`.
- * The first time you call `pthread_once` with an argument of `once_control`, it invokes `init_routine`, which is a function with no input arguments that returns nothing.
- * Subsequent calls to `pthread_once` with the same `once_control` variable do nothing.
- * The `pthread_once` function is useful whenever you need to dynamically initialize global variables that are shared by multiple threads.

Thread Execution Model



- ★ The execution model for multiple threads is similar in some ways to the execution model for multiple processes.
- ★ Each process begins life as a single thread called the main thread.
- ★ At some point, the main thread creates a peer thread, and from this point in time the two threads run concurrently.
- ★ Eventually, control passes to the peer thread via a context switch, because the main thread executes a slow system call such as read or sleep, or because it is interrupted by the system's interval timer.

- The peer thread executes for a while before control passes back to the main thread, and so on.



Thread Execution Model

- ★ Thread execution differs from processes in some important ways.
- ★ Because a thread context is much smaller than a process context, a thread context switch is faster than a process context switch.
- ★ Another difference is that threads, unlike processes, are not organized in a rigid parent-child hierarchy.
- ★ The threads associated with a process form a pool of peers, independent of which threads were created by which other threads.
- ★ The main thread is distinguished from other threads only in the sense that it is always the first thread to run in the process.
- ★ The main impact of this notion of a pool of peers is that a thread can kill any of its peers, or wait for any of its peers to terminate. Further, each peer can read and write the same shared data.

The Pthreads "hello, world" Program

```
/*  
 * hello.c - Pthreads "hello, world" program  
 */  
void *thread(void *vargp);  
  
int main()  
{  
    pthread_t tid;  
    pthread_create(&tid, NULL, thread, NULL);  
    pthread_join(tid, NULL);  
    exit(0);  
}
```

hello.c

Thread ID

Thread attributes
(usually NULL)

Thread routine

Thread arguments
(void *p)

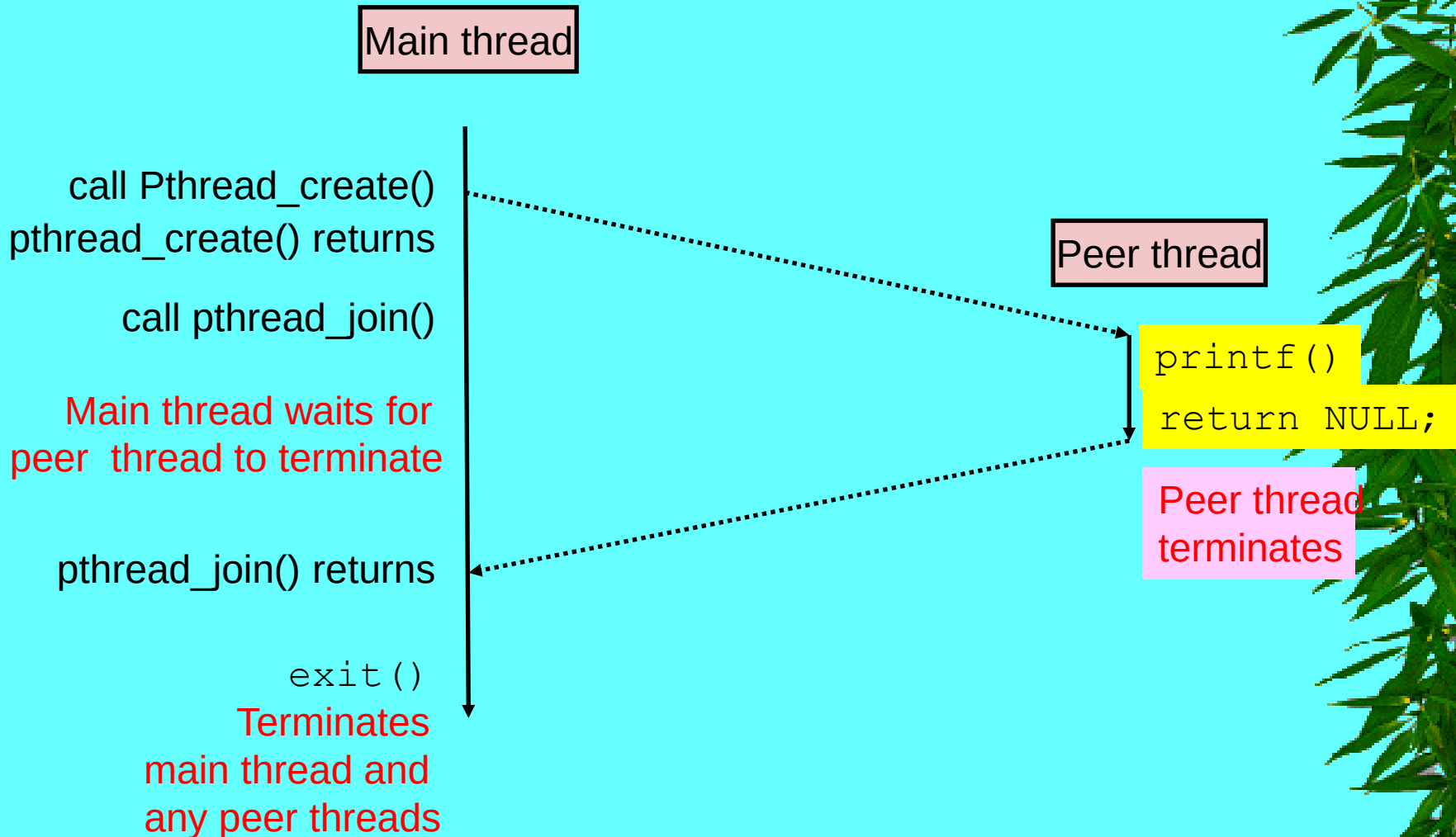
Return value
(void **p)

```
void *thread(void *vargp) /* thread routine */  
{  
    printf("Hello, world!\n");  
    return NULL;  
}
```

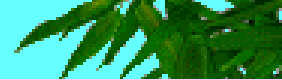
hello.c

65

Execution of Threaded “hello, world”



Thread Execution Model



- * Figure shows a simple Pthreads program.
- * The main thread creates a peer thread and then waits for it to terminate.
- * The peer thread prints “Hello, world!\n” and terminates.
- * When the main thread detects that the peer thread has terminated, it terminates the process by calling exit.
- * The code and local data for a thread is encapsulated in a thread routine.
- * As shown by the prototype in line 2, each thread routine takes as input a single generic pointer and returns a generic pointer.
- * If you want to pass multiple arguments to a thread routine, then you should put the arguments into a structure and pass a pointer to the structure.
- * Similarly, if you want the thread routine to return multiple arguments, you can return a pointer to a structure.
- * Line4 marks the beginning of the code for the main thread.
- * The main thread declares a single local variable tid, which will be used to store the thread ID of the peer thread(line6).
- * The main thread creates a new peer thread by calling the pthread_create function(line7).
- * When the call to pthread_create returns, the main thread and the newly created peer thread are running concurrently, and tid contains the ID of the new thread.
- * The main thread waits for the peer thread to terminate with the call to pthread_join in line 8.
- * Finally, the main thread calls exit (line9), which terminates all threads(in this case just the main thread) currently running in the process.
- * Lines 12–16 define the thread routine for the peer thread. It simply prints a string and then terminates the peer thread by executing the return statement in line 15.

Posix Threads



- ✱ Posix threads (Pthreads) is a standard interface for manipulating threads from C programs.
- ✱ It was adopted in 1995 and is available on most Unix systems.
- ✱ Pthreads defines about 60 functions that allow programs to create, kill, and reap threads, to share data safely with peer threads, and to notify peers about changes in the system state.

```
/* hello.c: The Pthreads “Hello, world!” program */
```

```
void *thread(void *vargp);
```

```
int main()
```

```
{
```

```
    pthread_t  tid;
```

```
    pthread_create(&tid, NULL, thread, NULL);
```

```
    pthread_join(tid, NULL);
```

```
    exit(0);
```

```
}
```

```
void *thread(void *vargp) /* Thread routine */
```

```
{
```

```
    printf("Hello, world!\n");
```

```
    return NULL;
```

```
}
```

SOC 2040

SYSTEM PROGRAMMING



END OF

WEEK 13 Lecture 2

EXCEPTIONAL CONTROL

FLOW

WISH YOU ALL THE BEST

14 May 2024

SPRING Semester 2024 @ Dr A R Naseer





INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

14 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG